

ROAMATE

백엔드 개발 기획서 및 작업지침서

외부 데이터·지도 담당 1 명 / 여행·AI 담당 1 명

개발 목표 현재 위치·날씨·사용자 상태를 분석하고, 검증된 관광지 후보와 실제 경로 데이터를 이용해 변경 가능한 여행 일정을 생성한다.

핵심 원칙 AI 는 후보 장소를 선택하고 설명만 생성한다. 장소·운영시간·날씨·이동시간은 외부 API 와 서버 검증 결과를 사용한다.

1. 개발 방향

1.1 완성해야 할 사용자 흐름

단계	처리	완료 조건
1	여행 조회	기존 일정·장소·시간을 불러온다.
2	현재 상황 수집	GPS, 현재 시각, 사용자 상태를 저장한다.
3	실시간 영향 분석	날씨·운영시간·남은 일정 충돌을 판정한다.
4	대체 후보 생성	관광공사 데이터에서 주변 후보를 조회한다.
5	경로 계산	네이버 Directions 로 후보별 이동시간을 계산한다.
6	AI 재설계	검증된 후보만 이용해 일정 순서와 변경 이유를 생성한다.
7	사용자 승인	변경 전후를 비교하고 승인된 경우에만 저장한다.

1.2 구현 범위

- 필수: 관광지 검색·상세·주변 조회, 여행·일정 CRUD, 날씨 조회, 자동차 경로 계산
- 필수: 현재 상황 저장, 일정 영향 분석, AI 재설계, 변경안 적용·거절, 변경 이력
- 제외: 대중교통·도보 내비게이션 복제, 예약·결제, 실시간 웨이팅, 음성 안내

1.3 구현 원칙

- 네이버 지도 화면은 프론트엔드에서 표시한다.
- 네이버 REST API Secret, 관광공사 서비스키, AI 키는 백엔드 환경변수로만 관리한다.
- 외부 API 응답을 그대로 반환하지 않고 내부 DTO 로 정규화한다.
- AI 에는 서버가 검증한 후보 관광지만 전달한다.
- AI 응답은 검증 후 임시 변경안으로 저장하고, 사용자 승인 전에는 기존 일정을 수정하지 않는다.
- 외부 API 장애 시 기존 일정은 유지하고 502 오류와 식별 가능한 오류 코드를 반환한다.

2. 역할 분담

담당	책임	패키지	브랜치
백엔드 A	관광공사·네이버 경로·날씨·주소·후보 점수	place, route, weather, location, external	feature/backend-data-map
백엔드 B	여행·일정·상황·영향 분석·Spring AI·변경안	trip, schedule, context, impact, replan, assistant	feature/backend-trip-ai

2.1 백엔드 A 완료 산출물

- Place API 및 KTO Client
- Naver Directions Client
- Reverse Geocoding Client
- Weather Client
- 후보 장소 검색·점수 로직
- 외부 API 예외 처리 및 Postman Collection

2.2 백엔드 B 완료 산출물

- Trip·Schedule·TravelContext·Replan 엔티티
- 여행·일정 CRUD
- Impact 분석 서비스
- Spring AI 구조화 응답 및 검증
- 변경안 적용·거절·이력 저장
- 통합 API 테스트

2.3 두 담당자 연결 규칙

백엔드 A가 제공하고 백엔드 B가 주입받을 Service 인터페이스

```
public interface PlaceSearchService {
    List<PlaceCandidate> searchNearby(
        double latitude,
        double longitude,
        CandidateSearchCondition condition
    );
}

public interface RouteCalculationService {
    RouteResult calculateDrivingRoute(
        Coordinate start,
        Coordinate goal,
        List<Coordinate> waypoints
    );
}

public interface WeatherQueryService {
    WeatherSnapshot getCurrentWeather(
        double latitude,
        double longitude
    );
}
```

금지 Controller가 다른 Controller를 직접 호출하지 않는다. 외부 API 호출은 반드시 백엔드 A의 Service 인터페이스를 통해 사용한다.

3. 내부 REST API 계약

Method	Endpoint	기능	담당
GET	/api/places	관광지 검색	A
GET	/api/places/nearby	현재 위치 주변 관광지	A
GET	/api/places/{contentId}	관광지 상세	A
POST	/api/routes/driving	자동차 경로 계산	A
GET	/api/locations/reverse-geocode	좌표→주소	A
GET	/api/weather/current	현재 날씨	A
POST	/api/trips	여행 생성	B
GET	/api/trips/{tripId}	여행·일정 조회	B
POST	/api/trips/{tripId}/schedules	일정 추가	B
PATCH	/api/trips/{tripId}/schedules/{scheduleId}	일정 수정	B
POST	/api/trips/{tripId}/context	현재 상황 저장	B
POST	/api/trips/{tripId}/impact/analyze	영향 분석	B
POST	/api/trips/{tripId}/replans	변경안 생성	B
POST	/api/trips/{tripId}/replans/{replanId}/apply	변경 적용	B
POST	/api/trips/{tripId}/replans/{replanId}/reject	변경 거절	B

3.1 공통 오류 응답

```
{
  "code": "EXTERNAL_API_ERROR",
  "message": "외부 서비스 호출에 실패했습니다.",
  "timestamp": "2026-07-20T14:20:00"
}
```

- 400: 좌표·시간·요청값 검증 실패
- 404: 여행·일정·관광지 없음
- 409: 이미 적용·거절된 변경안
- 502: 관광공사·네이버·날씨·AI 외부 호출 실패
- 500: 서버 내부 오류

4. 백엔드 A 작업지침 - 외부 데이터·지도

4.1 관광지 검색

프론트와 백엔드 B가 공통으로 사용할 관광지 응답 DTO

```
public record PlaceResponse(  
    String contentId,  
    String contentTypeId,  
    String title,  
    String address,  
    double latitude,  
    double longitude,  
    String imageUrl,  
    boolean indoor,  
    String openingHours,  
    String closedDay,  
    String source  
) {}
```

- mapX는 longitude, mapY는 latitude로 변환한다.
- 좌표가 없는 항목은 제외하고 이미지가 없으면 null을 반환한다.
- 기본 목록 + 공통 상세 + 소개 상세를 결합한 뒤 PlaceResponse로 반환한다.
- KTO 응답 원문은 Controller 밖으로 노출하지 않는다.

주변 관광지 API Controller 예시

```
@GetMapping("/api/places/nearby")  
public List<PlaceResponse> nearby(  
    @RequestParam double latitude,  
    @RequestParam double longitude,  
    @RequestParam(defaultValue = "5000") int radius,  
    @RequestParam(required = false) Boolean indoor  
) {  
    return placeService.searchNearby(  
        latitude, longitude, radius, indoor  
    );  
}
```

4.2 네이버 자동차 경로

```
public record RouteRequest(  
    Coordinate start,  
    Coordinate goal,  
    List<Coordinate> waypoints,  
    String option  
) {  
    public record Coordinate(  
        double latitude,  
        double longitude  
    ) {}  
}
```

Directions 호출 핵심 코드

```
URI uri = UriComponentsBuilder  
    .fromHttpUrl(directionsUrl)  
    .queryParam("start", start.longitude() + "," + start.latitude())  
    .queryParam("goal", goal.longitude() + "," + goal.latitude())  
    .queryParam("waypoints", waypointValue)  
    .queryParam("option", "trafast")
```

```
.build()
    .toUri();

return restClient.get()
    .uri(uri)
    .header("x-ncp-apigw-api-key-id", clientId)
    .header("x-ncp-apigw-api-key", clientSecret)
    .retrieve()
    .body(NaverRouteResponse.class);
```

- 경유지는 최대 5 개로 제한한다.
- 네이버 좌표 형식은 경도,위도 순서다.
- 응답 path 를 프론트용 longitude/latitude 배열로 변환한다.

4.3 날씨·주소·후보 점수

야외 일정 위험 판정 기준

```
public boolean calculateOutdoorRisk(
    String condition,
    double temperature,
    double windSpeed,
    int rainProbability
) {
    return switch (condition) {
        case "RAIN", "THUNDERSTORM", "SNOW" -> true;
        default -> temperature >= 32
            || windSpeed >= 10
            || rainProbability >= 70;
    };
}
```

대체 후보 정렬 점수

```
public double calculateScore(PlaceCandidate place) {
    double score = 100.0;
    score -= place.travelMinutes() * 1.5;
    if (place.indoor()) score += 20;
    if (place.openNow()) score += 25;
    else score -= 100;
    score += place.preferenceScore() * 15;
    score += place.dataCompleteness() * 10;
    return score;
}
```

4.4 백엔드 A 완료 기준

- 관광지 검색·주변·상세 API 가 동일한 PlaceResponse 규격을 반환한다.
- Directions 요청이 실제 거리·소요시간·path 를 반환한다.
- 현재 좌표를 주소로 변환한다.
- 날씨 API 가 outdoorRisk 를 반환한다.
- 후보 장소를 점수순으로 최대 10 개 반환한다.
- 외부 API 타임아웃·4xx·5xx 를 EXTERNAL_API_ERROR 로 변환한다.

5. 백엔드 B 작업지침 - 여행·일정·AI

5.1 핵심 데이터 모델

엔티티	필수 필드	용도
Trip	id, title, regionCode, startDate, endDate, status	여행 단위
Place	contentId, title, lat, lng, openingHours, indoor	관광지 캐시
Schedule	tripId, placeId, order, start/end, travelMinutes, mustVisit	일정 항목
TravelContext	tripId, location, currentTime, userCondition, weather	현재 상황
Replan	tripId, contextId, summary, status	변경안 헤더
ReplanItem	replanId, action, placeId, order, start/end, reason	변경안 상세

5.2 일정 영향 분석

```
public ImpactResult analyze(
    List<Schedule> remaining,
    WeatherSnapshot weather,
    UserCondition condition,
    LocalDateTime now
) {
    List<AffectedSchedule> affected = remaining.stream()
        .filter(s ->
            (weather.outdoorRisk() && !s.getPlace().isIndoor())
            || s.arrivesAfterClosingTime()
        )
        .map(AffectedSchedule::from)
        .toList();

    Severity severity = !affected.isEmpty()
        ? Severity.HIGH
        : condition == UserCondition.TIRED
        ? Severity.MEDIUM
        : Severity.LOW;

    return new ImpactResult(
        severity != Severity.LOW,
        severity,
        affected
    );
}
```

5.3 재설계 처리 순서

- Trip 과 현재 시각 이후 Schedule 조회
- WeatherQueryService 로 현재 날씨 조회
- 영향받은 일정과 기존 contentId 추출
- PlaceSearchService 로 주변 후보 조회
- RouteCalculationService 로 후보별 이동시간 계산
- 점수 상위 10 개 후보를 AI 입력으로 제한
- AI 구조화 응답 수신 및 검증
- Replan·ReplanItem 을 PENDING 상태로 저장
- 변경 전후 비교 결과 반환

5.4 Spring AI 코드 방향

AI 출력은 문자열이 아니라 Java DTO 로 받는다.

```
public record AiReplanResult(  
    String summary,  
    List<AiScheduleItem> schedules,  
    List<AiChangeItem> changes  
) {}  
  
public record AiScheduleItem(  
    String contentId,  
    int orderNumber,  
    LocalDateTime startTime,  
    LocalDateTime endTime,  
    String reason  
) {}
```

Spring AI 호출 예시

```
return chatClient.prompt()  
    .system("""  
        제공된 후보 관광지만 사용한다.  
        contentId, 운영시간, 이동시간을 만들지 않는다.  
        필수 방문지는 유지하고 시간 충돌을 만들지 않는다.  
        """)  
    .user(prompt)  
    .call()  
    .entity(AiReplanResult.class);
```

5.5 AI 응답 검증

```
public void validate(  
    AiReplanResult result,  
    Set<String> allowedContentIds,  
    Set<String> mustVisitIds  
) {  
    for (AiScheduleItem item : result.schedules()) {  
        if (!allowedContentIds.contains(item.contentId())) {  
            throw new InvalidAiResponseException(  
                "허용되지 않은 관광지"  
            );  
        }  
    }  
    validateMustVisit(result, mustVisitIds);  
    validateTimeOverlap(result.schedules());  
    validateOpeningHours(result.schedules());  
    validateTravelTime(result.schedules());  
}
```

- 1 차 검증 실패 시 프롬프트에 오류 원인을 포함해 한 번만 재시도한다.
- 2 차 실패 시 규칙 기반 대체안 또는 기존 일정 유지 결과를 반환한다.
- 검증되지 않은 AI 결과는 DB 에 저장하지 않는다.

5.6 변경안 적용

```
@Transactional
public TripResponse apply(Long tripId, Long replanId) {
    Replan replan = replanRepository.findPending(tripId, replanId)
        .orElseThrow(ReplanNotApplicableException::new);
    scheduleService.replaceRemainingSchedules(tripId, replan.toSchedules());
    replan.apply();
    return tripQueryService.getTrip(tripId);
}
```

6. 프론트엔드 연동 규격

6.1 현재 상황 저장

```
POST /api/trips/1/context

{
  "latitude": 35.1532,
  "longitude": 129.1187,
  "currentTime": "2026-07-20T14:20:00",
  "currentScheduleId": 3,
  "userCondition": "TIRED"
}
```

6.2 변경안 생성

```
POST /api/trips/1/replans

{
  "contextId": 41,
  "options": {
    "preferIndoor": true,
    "minimizeWalking": true,
    "preserveMustVisit": true,
    "maximumPlaces": 4
  }
}
```

6.3 변경안 응답

```
{
  "replanId": 22,
  "summary": "비와 피로 상태를 반영해 야외 일정 1 곳을 교체했습니다.",
  "before": { "travelMinutes": 96, "outdoorCount": 2, "endTime": "20:40" },
  "after": { "travelMinutes": 68, "outdoorCount": 0, "endTime": "20:10" },
  "changes": [{
    "action": "REPLACE",
    "oldPlaceName": "해안 산책로",
    "newPlaceName": "부산현대미술관",
    "reason": "우천 영향을 받지 않고 기존 동선과 가깝습니다."
  }],
  "proposedSchedules": []
}
```

프론트 처리 proposedSchedules 를 즉시 현재 일정으로 덮어쓰지 않는다. 비교 화면을 표시한 뒤 apply API 성공 응답으로 최종 화면을 갱신한다.

7. 작업 일정 및 완료 기준

일차	백엔드 A	백엔드 B
1일	외부 API 키·Postman 검증, DTO 확정	프로젝트·DB·엔티티·샘플 Trip
2일	관광지 검색·주변·상세	여행·일정 CRUD
3일	Directions·주소·날씨	Context·Impact 분석
4일	후보 점수·외부 예외 처리	Spring AI·응답 검증
5일	경로 비교·통합 지원	Replan 저장·적용·거절
6일	Postman Collection·문서	테스트·오류 처리
7일	프론트 통합 수정	프론트 통합 수정

7.1 최종 통합 테스트

- GET /api/trips/{id}로 일정 조회
- POST /context 로 현재 위치·상태 저장
- 날씨·주소·영향 분석 결과 확인
- 주변 대체 후보 및 후보별 실제 자동차 이동시간 확인
- POST /replans 로 변경 전후 비교 응답 확인
- apply 호출 후 기존 일정이 승인된 일정으로 교체되는지 확인
- reject 호출 후 기존 일정이 유지되는지 확인
- 외부 API 장애 시 기존 일정 유지 및 502 오류 확인

7.2 개발 완료 판정

완료 프론트 화면에서 현재 일정 → 실시간 변수 감지 → 변경안 비교 → 사용자 승인 → 지도·일정 갱신의 전체 흐름이 한 번에 작동하면 완료로 판정한다.

8. 공식 문서 확인 항목

- 네이버 Maps JavaScript API v3: ncpKeyId 로 지도 SDK 로드
- 네이버 Maps REST API: API Gateway 인증 후 경로·주소 변환 사용
- Directions 5: 자동차 경로, 경유지 최대 5개
- 한국관광공사 국문 관광정보 OpenAPI: 지역·위치·키워드·상세·이미지 정보
- Spring AI: ChatClient 및 구조화 출력 변환

구현 전 확인 외부 API URL·요청 헤더·operation 이름은 발급받은 콘솔 및 공식 명세의 최신 값을 기준으로 application.yml 에 입력한다.